

World-Model Based Reinforcement Learning for Hockey: DreamerV3 with Adaptive Opponent Curriculum

Carl Kueschall
University of Tübingen

Winter Semester 2025/26

1 Introduction

Note: Originally a two-person project; co-author exmatriculated. Work completed by remaining author.

The hockey environment [8] presents a challenging continuous control problem in an adversarial setting. Two agents compete in an air-hockey game with 18-dimensional state observations encoding positions, velocities, and puck state, while each player controls 4 continuous action dimensions. Episodes last up to 250 timesteps with extremely sparse rewards: +10 for scoring, −10 for conceding, and 0 otherwise. This sparsity makes credit assignment difficult—agents must learn complex behaviors (positioning, puck control, shooting, defending) from reward signals that occur only at episode boundaries.

I implement DreamerV3 [4], which learns a world model from real rollouts and trains the policy in imagined latent trajectories. Starting from NaturalDreamer¹, I add hockey-specific components: Two-Hot Symlog heads, DreamSmooth [10], sparse-event loss weighting, PFSP self-play [9], and an adaptive opponent curriculum. The main practical result is that late gains are driven more by recursive pool curation than by further fixed-bot training.

2 Methods

2.1 DreamerV3: World-Model Based RL

2.1.1 World Model Architecture

DreamerV3 [4] uses an RSSM with deterministic and stochastic dynamics. The latent state is $s_t = (h_t, z_t)$ with GRU memory $h_t \in \mathbb{R}^{256}$ and categorical z_t (16 variables, 16 classes each). Dynamics: $h_t = \text{GRU}(h_{t-1}, z_{t-1}, a_{t-1})$; prior $\hat{z}_t \sim p_\theta(z_t|h_t)$; posterior $z_t \sim q_\phi(z_t|h_t, \text{Enc}(o_t))$. Training uses KL with free nats, plus a continuation head for non-terminal prediction and imagination-time return weighting (Figure 1).

2.1.2 Key Modifications for Hockey

Table 1 summarizes our domain-specific modifications. **DreamSmooth** [10] propagates goal rewards backward: $\tilde{r}_t = \alpha r_t + (1 - \alpha)\tilde{r}_{t+1}$ with $\alpha = 0.5$, addressing temporal credit assignment for goals at episode boundaries. In addition, I use inverse-frequency weighting for sparse reward events ($|r| > 1$) in reward-head training, capped for stability, to prevent rare goal transitions from being gradient-diluted by the dominant near-zero reward mass.

¹<https://github.com/InexperiencedMe/NaturalDreamer>

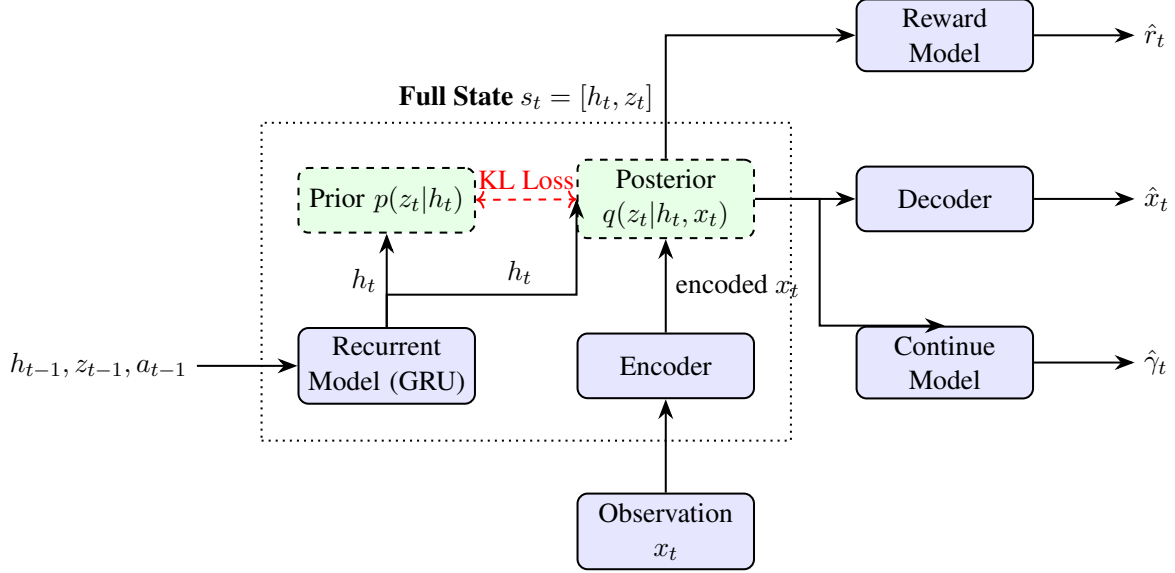


Figure 1: DreamerV3 world-model training (reality mode): posterior inference from observations, prior matching via KL, and decoder/reward/continue heads from latent state; adapted from [4, 7].

Modification	Problem Addressed	Implementation
Two-Hot Symlog	Gaussian reward/value heads underfit sparse multimodal targets	255 bins in symlog space
DreamSmooth	Temporal credit assignment	$\tilde{r}_t = \alpha r_t + (1 - \alpha)\tilde{r}_{t+1}$, $\alpha = 0.5$
Sparse-event loss weighting	Rare goal events under-weighted in reward loss	Inverse-frequency weighting for $ r > 1$, clipped for stability
Anchor Opponents	Prevent catastrophic forgetting	10% weak + 10% strong in late phase
Self-Play + PFSP [9]	Limited diversity	48-checkpoint pool, variance-weighted PFSP, periodic insertion

Table 1: Key modifications from baseline DreamerV3 for the hockey domain.

2.1.3 Recursive Opponent-Pool Management

The final training policy is controlled by opponent sampling rather than reward shaping, with $\mathbb{P}(\text{anchor}) = \rho$ and $\mathbb{P}(\text{self-play}) = 1 - \rho$. In late runs we set $\rho = 0.2$, yielding an empirical distribution close to 80/10/10 (self-play/weak/strong). This keeps fixed-bot competence while maximizing exposure to diverse, hard opponents. In league runs, PFSP sampling [9] is further blended with a static checkpoint prior from curated league weights, using exponent α (`self_play_prior_alpha`) so higher-quality checkpoints are sampled earlier without disabling PFSP adaptation.

To avoid selecting brittle checkpoints, I rank candidates with robustness-aware metrics from cross-play: $\text{RobustScore} = 0.60 \cdot \text{CP}_{\min} + 0.25 \cdot \text{Fixed}_{\text{combined}} + 0.15 \cdot \text{CP}_{\text{mean}}$, and also use CP_{q20} (20th percentile) for non-transitive settings.

2.1.4 Behavior Learning

Actor: Tanh-squashed Gaussian with $\sigma_{\min} = 0.1$ plus mean regularization (`mean_reg_scale`) to reduce tanh-saturation drift. **Critic:** Two-Hot Symlog with slow-critic EMA regularization ($\tau = 0.98$).

Returns: TD(λ), $\lambda = 0.95$, $\gamma = 0.997$, with percentile value normalization (5th/95th). **Actor loss:**

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}_{s \sim \text{imagine}} \left[\frac{V^\lambda(s) - V(s)}{S} + \eta H[\pi(\cdot|s)] \right], \text{ fixed } \eta = 3 \times 10^{-4}.$$

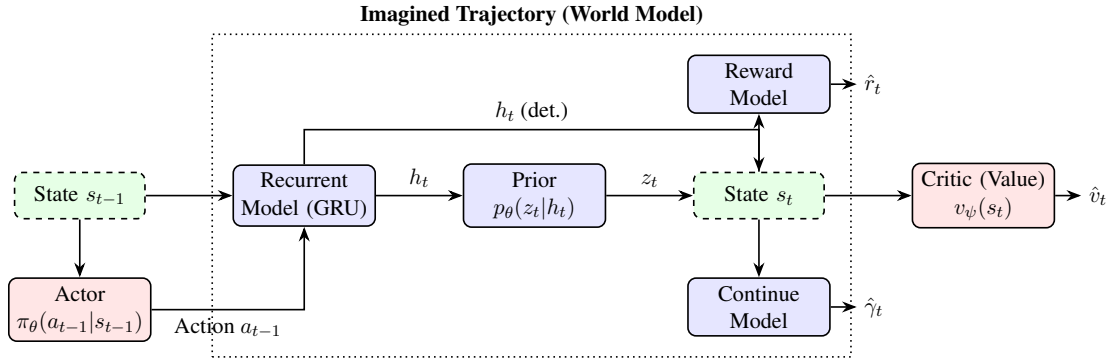


Figure 2: DreamerV3 behavior training (imagination mode): actor and critic are trained on prior-only latent rollouts (no observation encoder in-loop); adapted from [4, 7].

Figure 2 shows the pure imagination loop used for policy optimization.

3 Experimental Evaluation

3.1 From 2-Phase Baseline to Adaptive 3-Stage Curriculum

The early setup (mixed fixed bots, then self-play) reached a strong 336k fixed-bot baseline. Final runs showed a clearer compute split (Table 2): fixed-bot training is necessary but saturates early; most late gains come from curated league self-play.

Stage	Share	Opponents	Outcome
A: Anchor competence	~20%	Weak + strong fixed bots	Base skills + stable world model.
B: Bootstrap self-play	~20%	Self-play + anchors	Adversarial adaptation; stronger mids (474k/556k).
C: Curated league self-play	~60%	80% league, 10% weak, 10% strong	Robustness and best late checkpoints (612k/634k).

Table 2: Practical training-time allocation learned during the final push.

Appendix Figure 6 gives the full flow. In a representative late run, sampling matched targets (self-play ≈ 0.80 , anchors ≈ 0.20), the pool grew to 48 opponents, and pool metrics reached 0.804 overall / 0.725 newest-third win rate.

3.2 Summit Cross-Play and Checkpoint Selection

Matrix cross-play against fixed bots plus a curated opponent bank improved over earlier baselines, while still showing non-transitivity [9, 6]. In practice, fixed-bot win rate saturates and becomes weakly informative; `CP_worst` and lower-quantile metrics (`CP_Q20`) are better late-stage selectors. I therefore used a two-stage automated shortlist pipeline (`eval_select_pool.py`): fast screening, then dense top- K refinement, followed by greedy quality+diversity selection. The resulting 21-checkpoint pool is listed in Appendix Table 3; Appendix Figure 8 shows its tradeoff view. Figure 3 shows the overlap-clipped single timeline built from four runs.

3.3 Ablation Diagnostics Beyond Aggregate Win Rate

Figures 4 and 5 are interpreted jointly, not by headline win rate alone. In this hockey setup, neither DreamSmooth nor Two-Hot produced a massive jump in `eval/combined_win_rate`; DreamerV3 already performs strongly on the sparse-reward task.

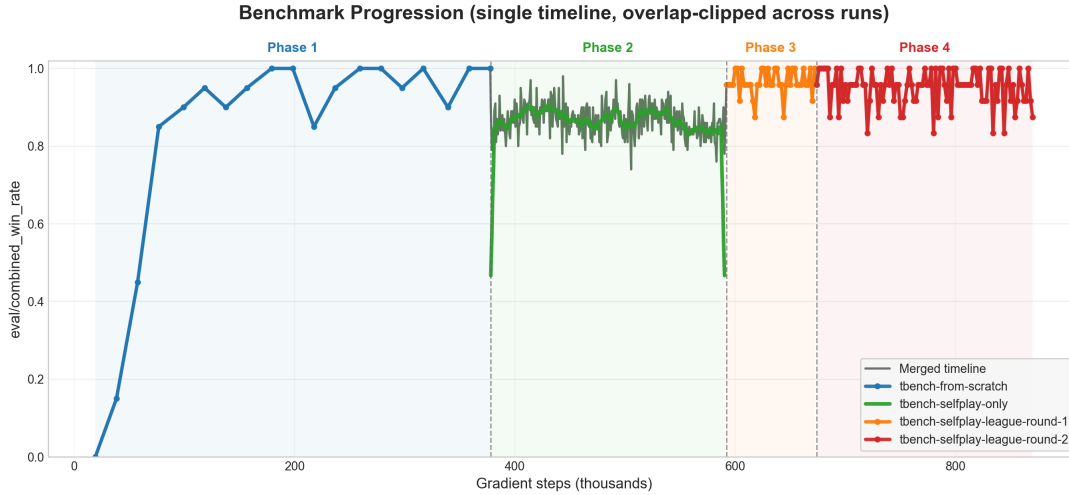


Figure 3: Single-timeline eval/combined_win_rate from four runs, overlap-clipped in gradient-step space; colors mark source phase.

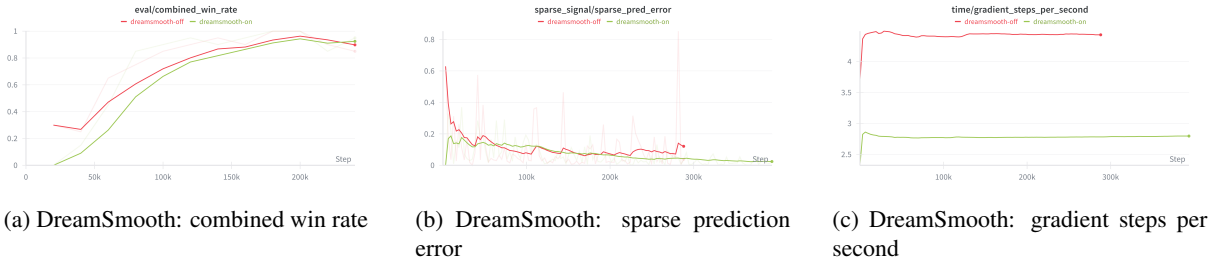


Figure 4: DreamSmooth ablation diagnostics beyond headline win rate.

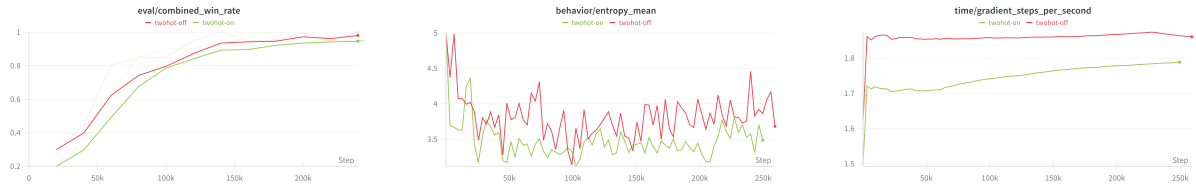
The diagnostics are still useful: DreamSmooth reduces sparse-prediction error but lowers throughput (Figures 4b,c). Two-Hot gives similar final win rate to Gaussian but shifts entropy behavior (Figure 5b), i.e., policy-calibration effects without a large headline gain. Sparse-event loss weighting is active in all runs (Section 2.1); it was part of the baseline, not isolated as a separate ablation.

4 Discussion

Key Findings and Lessons Learned. DreamSmooth and Two-Hot did not strongly change headline win rate in this single-seed setting, but they behaved as intended: DreamSmooth improved sparse-signal prediction, Two-Hot changed entropy behavior, and sparse-event weighting improved rare-goal gradient allocation. The dominant late-stage lever was opponent curriculum design: once fixed-bot competence saturated, extra compute was best spent on curated league self-play (about 20/20/60 for anchor/bootstrap/league). Recursive pool management [9] (checkpoint insertion, PFSP, matrix-based curation) produced the largest gains.

Post-hoc algorithm-fit diagnosis. A targeted literature review suggests this result is plausible: DreamerV3 is often strongest when sample efficiency and high-dimensional modeling dominate, while low-dimensional, fast-simulator continuous control can favor strong model-free baselines (TD3/SAC) in asymptotic score [4, 2, 3]. This aligns with hockey and newer evidence that architecture/planning choices matter strongly in continuous control (e.g., TD-MPC2 vs. DreamerV3) [5].

Limitations and Future Work. Many matrix snapshots still use modest budgets (often 30 episodes/matchup), so ranking among close late checkpoints has variance. Non-transitivity remains visible [1, 6], some legacy checkpoints required compatibility filtering, and sparse-event weighting was not isolated with a



(a) Two-Hot: combined win rate

(b) Two-Hot: entropy mean

(c) Two-Hot: gradient steps per second

Figure 5: Two-Hot vs. Gaussian-head diagnostics (performance, entropy behavior, and throughput).

multi-seed ablation. Next steps are higher-budget bidirectional matrices (100+ episodes/matchup, multiple seeds), continuously refreshed multi-seed leagues, and online tournament feedback in promotion criteria.

5 AI Usage Declaration

My AI-use philosophy had three layers. First, I used agentic systems (primarily Claude Code) for discussion, debate, decision support, concept learning, and targeted paper discovery. Second, I used AI autocomplete for implementing core logic (agent internals, training loop, architecture-level essentials), followed by manual review and verification. Third, I used Claude Code for low-learning-value “dirty work” (repetitive scripting, formatting, and pipeline glue). The decision matrix is shown in Appendix Figure 7. The report text itself was written by the author; AI was only used for grammar/spell checking (e.g., DeepL), in line with course guidance. Detailed prompts for larger code-generation tasks are provided in `AI_PROMPTS.md` in the submitted code folder.

References

- [1] D. Bairamian, P. Marcotte, J. Romoff, G. Robert, and D. Nowrouzezahrai. Minimax exploiter: A data efficient approach for competitive self-play. *arXiv preprint arXiv:2311.17190*, 2023.
- [2] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, pages 1587–1596. PMLR, 2018.
- [3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870. PMLR, 2018.
- [4] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2024.
- [5] N. Hansen, H. Su, and X. Wang. Td-mpc2: Scalable, robust world models for continuous control. In *The Twelfth International Conference on Learning Representations*, 2024.
- [6] R. Huang, X. Wu, H. Yu, Z. Fan, H. Fu, Q. Fu, and W. Yang. A robust and opponent-aware league training method for starcraft ii. In *Advances in Neural Information Processing Systems*, 2023.
- [7] InexperiencedMe. Naturaldreamer: A readable implementation of dreamerv3. <https://github.com/InexperiencedMe/NaturalDreamer>, 2024.
- [8] G. Martius, P. Koley, M. Zhobro, and S. Ramen. Hockey environment. <https://github.com/martius-lab/hockey-env>, 2024.
- [9] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.
- [10] V. Wu, J. Aslanides, J. Ba, and D. Hafner. Dreamsmooth: Improving model-based reinforcement learning via reward smoothing. *arXiv preprint arXiv:2311.01450*, 2023.

A Appendix

A.1 Training Flow Visuals

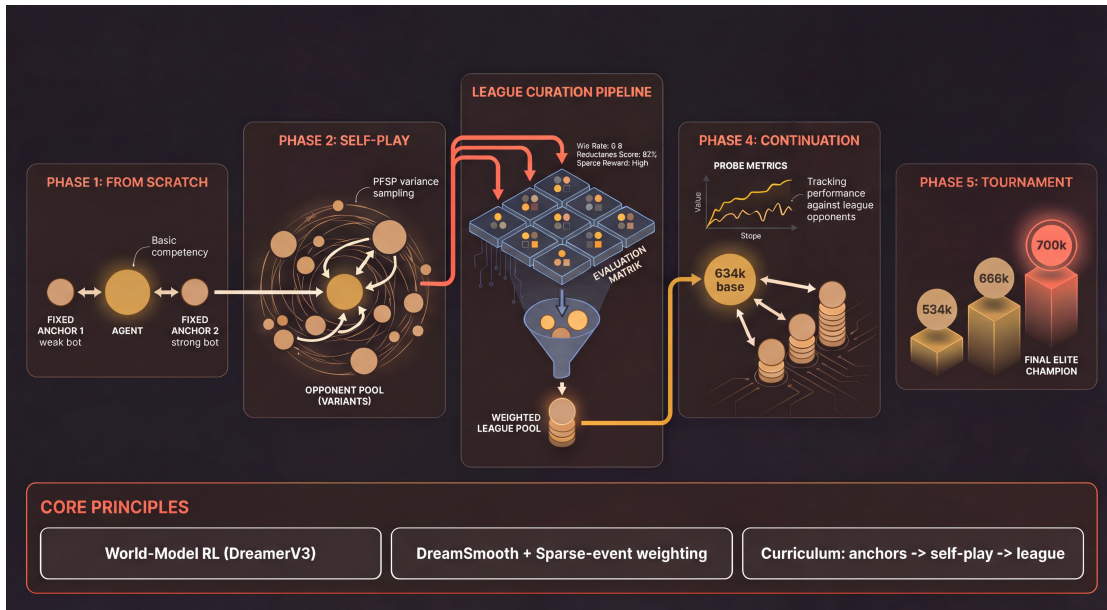


Figure 6: Holistic training-phase visual from scratch to final tournament checkpoint selection.

A.2 AI-Use Decision Matrix

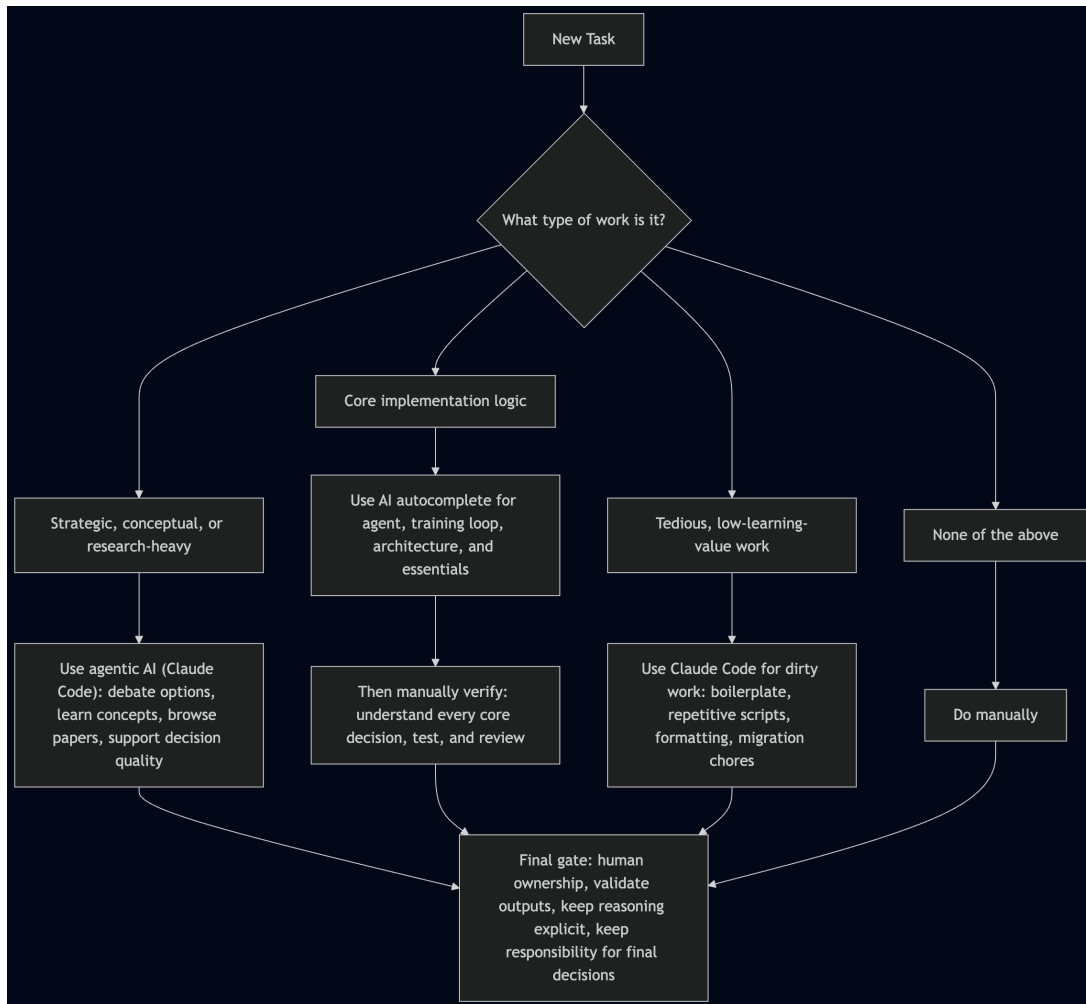


Figure 7: AI-use decision matrix (rendered chart).

A.3 Full League-Pool Ranking

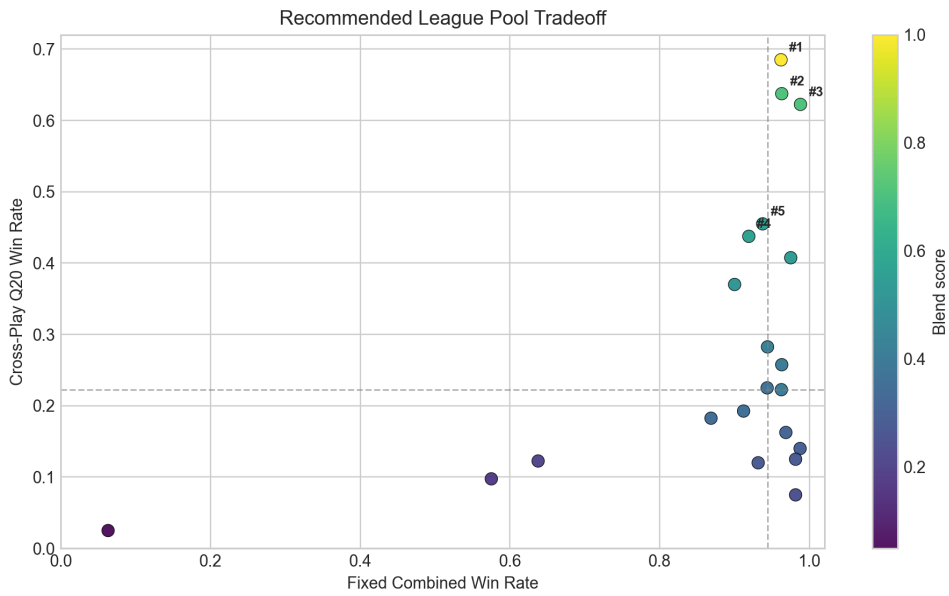


Figure 8: Recommended-pool tradeoff view. The x-axis shows fixed-bot strength and the y-axis shows cross-play robustness (CP_Q20); color indicates blend score used for sampling priority.

Table 3: Full 21-checkpoint ranking used to build the curated league pool.

Rank	Checkpoint	Blend	Quality	Diversity	CP Q20	CP Worst	CP Mean	Fixed Combined
1	634k.pth	1.0000	0.7481	1.0000	0.6850	0.5250	0.7712	0.9620
2	612k.pth	0.7111	0.7101	0.0054	0.6375	0.2750	0.7325	0.9630
3	622k.pth	0.7093	0.7087	0.0039	0.6225	0.4625	0.7481	0.9880
4	results_checkpoints__weak_seed42_20260224_074312_546k.pth	0.5678	0.5708	0.0207	0.4375	0.2375	0.6819	0.9190
5	556k.pth	0.5552	0.5572	0.0277	0.4550	0.0125	0.5744	0.9375
6	results_checkpoints__weak_seed42_20260224_013546_558k.pth	0.5496	0.5557	0.0118	0.4075	0.3250	0.6600	0.9750
7	results_checkpoints_summit-local-league80-20260224_010504_ weak_seed42_20260224_010505_534k.pth	0.5165	0.5245	0.0115	0.3700	0.2000	0.6700	0.9000
8	results_checkpoints__weak_seed42_20260223_031428_456k.pth	0.4258	0.4306	0.0454	0.2825	0.2000	0.4781	0.9440
9	results_checkpoints_DreamerV3-mixed-selfplay-seed42_weak_ seed42_20260124_144802_220k.pth	0.4090	0.4177	0.0330	0.2225	0.0625	0.5594	0.9625
10	results_checkpoints__weak_seed42_20260223_040357_534k.pth	0.4070	0.4202	0.0143	0.2575	0.0375	0.4850	0.9630
11	results_checkpoints__weak_seed42_20260223_031358_406k.pth	0.3673	0.3815	0.0190	0.2250	0.1000	0.4200	0.9435
12	results_checkpoints__weak_seed42_20260224_074312_480k.pth	0.3548	0.3700	0.0181	0.1925	0.0500	0.4706	0.9120
13	results_checkpoints__weak_seed42_20260224_040000_476k.pth	0.3467	0.3629	0.0154	0.1825	0.0375	0.4925	0.8685
14	results_checkpoints_dreamer_weak_seed43_20260125_133552_ 240k.pth	0.3144	0.3284	0.0320	0.1625	0.0000	0.3425	0.9685
15	results_checkpoints_ balanced-finetune-DreamerV3-final-finetune-seed43_weak_ seed43_20260125_230625_266k.pth	0.2995	0.3184	0.0147	0.1400	0.0125	0.3450	0.9875
16	357k_0130_193448.pth	0.2796	0.2974	0.0243	0.1200	0.0375	0.3425	0.9315
17	results_checkpoints_dreamer_weak_seed43_20260125_133536_ 250k.pth	0.2788	0.2996	0.0114	0.1250	0.0125	0.3094	0.9815
18	results_checkpoints_ balanced-finetune-DreamerV3-final-finetune-seed43_weak_ seed43_20260125_215906_296k.pth	0.2445	0.2653	0.0189	0.0750	0.0000	0.2925	0.9815
19	results_checkpoints_DreamerV3-dreamsmooth-seed42_weak_ seed42_20260122_000848_120k.pth	0.2058	0.2187	0.0615	0.1225	0.0375	0.1981	0.6375
20	results_checkpoints_DreamerV3-small-weak-seed43_weak_seed43_ 20260120_122246_50k.pth	0.1772	0.1952	0.0462	0.0975	0.0500	0.2019	0.5750
21	dreamer_weak_seed42_20260124_141543_final.pth	0.0490	0.0385	0.1960	0.0250	0.0125	0.0563	0.0630